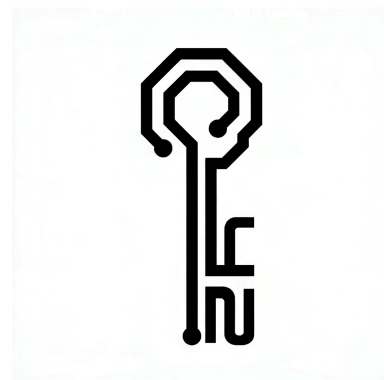


Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра № 42 «Криптология и кибербезопасность»



ОТЧЕТ

**О выполнении лабораторной работы № 4
«Технология OpenMP. Особенности настройки»**

Студент: Доценко В. А.

Группа: Б23-505

Преподаватель: Куприяшин М.А.

Москва — 2025

Оглавление

1. Цель работы	3
2. Рабочая среда	3
3. Описание рассмотренных в работе опций и директив	3
4. Вывод параметров OpenMP и текущих настроек среды	4
5. Алгоритм с использованием <code>omp_set_lock</code>	4
6. Разработанные программные коды	5
7. Результаты вычислительных экспериментов для алгоритма сортировки Шелла	7
7.1. Дополнительное задание	9
8. Заключение	11

1. Цель работы

Изучить основные настройки среды OpenMP и связанные с ними возможности.

2. Рабочая среда

Модель процессора: Apple M3.

Количество ядер / потоков: 8 / 8.

Объем оперативной памяти: 24 ГБ.

Разрядность и версия ОС: macOS Sequoia 15.6.

Среда разработки: VS Code, clang 16.0.0.

Версия OpenMP: 4.0 (_OPENMP = 202011).

3. Описание рассмотренных в работе опций и директив

omp_get_num_procs() – возвращает количество процессоров (логических CPU), доступных программе.

omp_get_max_threads() - возвращает максимальное количество потоков, которые OpenMP использует по умолчанию в параллельной области.

Опция dynamic Позволяет OpenMP динамически изменять количество потоков в параллельной области в зависимости от загрузки системы.
omp_get_dynamic() - возвращает 1, если динамическое управление потоками включено, иначе 0.

omp_get_wtick() - возвращает разрешение системного таймера, т.е. минимальный шаг измерения времени (в секундах).

omp_get_nested() – возвращает 1, если вложенные параллельные области поддерживаются, иначе 0.

omp_get_max_active_levels() - возвращает максимальное количество уровней активных параллельных областей, разрешённых в программе OpenMP.

OpenMP поддерживает несколько политик планирования:

- **Static**: итерации делятся заранее, ещё до начала выполнения, каждый поток получает фиксированный блок итераций, нет накладных расходов на планирование.

- **Dynamic**: итерации выдаются по мере освобождения потоков, после завершения текущего блока поток получает следующий.
- **Guided**: похож на dynamic, но размер блока уменьшается с течением времени, сначала выдаются крупные блоки, затем всё меньшие.
- **Auto**: решение о распределении итераций принимает реализация OpenMP
`omp_get_schedule()` - позволяет получить текущие настройки.

4. Вывод параметров OpenMP и текущих настроек среды

Получение версии OpenMP, кол-ва доступных процессоров процессоров, потоков, состояния опции dynamic, разрешения таймера и текущих настроек среды.

Состояние среды по умолчанию:

OpenMP version macro: 202011

OpenMP standard date: 2020-11

Number of processors: 8

Max threads: 8

Dynamic threads enabled: 0

Timer resolution: 1e-06 seconds

Nested parallelism enabled: 0

Max active levels: 1

Schedule type: static

Chunk size: 0

5. Алгоритм с использованием `omp_set_lock`

`omp_set_lock()` - захватывает указанный `omp_lock_t` и блокирует вызывающий поток до тех пор, пока блокировка не станет доступной (после `omp_unset_lock()`)

В разработанном алгоритме несколько потоков OpenMP параллельно формируют выборку случайных значений и обновляют общую гистограмму, состоящую из NUM_BINS корзин. Обновление корзины гистограммы представляет собой небезопасную по отношению к гонкам последовательность операций чтения, модификации и записи (read-modify-write). Поскольку

эта последовательность не является атомарной, одновременное обращение нескольких потоков к одной и той же корзине приводит к race condition, проявляющейся в потере инкрементов и, как следствие, в неверном суммарном результате.

Для корректной параллельной работы используется массив явных блокировок `omp_lock_t`, содержащий по одной блокировке для каждой корзины гистограммы. Применение `omp_set_lock()` / `omp_unset_lock()` гарантирует корректность обновления каждой конкретной корзины. Их отсутствие приводит к несогласованным конкурентным записям, в результате чего итоговая сумма элементов гистограммы систематически не совпадает с ожидаемым значением, что наглядно демонстрирует наличие гонки данных.

6. Разработанные программные коды

```
#include <iostream>
#include <vector>
#include <random>
#include <omp.h>

int main() {
    const int NUM_BINS = 10;
    const long NUM_ITEMS = 10'000'000;
    std::vector<long> histogram(NUM_BINS, 0);

    std::vector<omp_lock_t> locks(NUM_BINS);
    for (int i = 0; i < NUM_BINS; ++i) omp_init_lock(&locks[i]);
    std::mt19937_64 rng;
    unsigned seed = static_cast<unsigned>(omp_get_thread_num()) ^ (unsigned)time(nullptr);
    rng.seed(seed);
    std::uniform_int_distribution<int> dist(0, NUM_BINS - 1);

    #pragma omp parallel
    {
        #pragma omp for schedule(static)
        for (long i = 0; i < NUM_ITEMS; ++i) {
            int bin = dist(rng);

            omp_set_lock(&locks[bin]);
            ++histogram[bin];
            omp_unset_lock(&locks[bin]);
        }
    }

    for (int i = 0; i < NUM_BINS; ++i) omp_destroy_lock(&locks[i]);

    for (int i = 0; i < NUM_BINS; ++i) {
        std::cout << "bin[" << i << "] = " << histogram[i] << "\n";
    }
}
```

```

    long sum = 0;
    for (auto v : histogram) sum += v;
    std::cout << "total = " << sum << " (expected " << NUM_ITEMS << ")\n";
    return 0;
}

```

Файл 1 – Параллельный алгоритм с использованием omp_lock_t

```

#include <omp.h>

void shell_sort_parallel(int *array, int size, int threads, int schedule_type, int chunk) {
    omp_sched_t kind;
    switch(schedule_type) {
        case 0: kind = omp_sched_static; break;
        case 1: kind = omp_sched_dynamic; break;
        case 2: kind = omp_sched_guided; break;
        case 3: kind = omp_sched_auto; break;
        default: kind = omp_sched_static; break;
    }
    omp_set_schedule(kind, chunk);

    for (int s = size / 2; s > 0; s /= 2) {
        #pragma omp parallel for num_threads(threads) schedule(runtime) shared(array, size,
s) default(none)
        for (int offset = 0; offset < s; ++offset) {
            for (int i = offset + s; i < size; i += s) {
                for (int j = i - s; j >= 0 && array[j] > array[j + s]; j -= s) {
                    int temp = array[j];
                    array[j] = array[j + s];
                    array[j + s] = temp;
                }
            }
        }
    }
}

```

Файл 2 – Код алгоритма сортировки Шелла из 3 лабораторной, добавлена параметризация параллелизма

```

#include <iostream>
#include <omp.h>

int main() {
    std::cout << "OpenMP version macro: " << _OPENMP << std::endl;

    int year = _OPENMP / 100;
    int month = _OPENMP % 100;
    std::cout << "OpenMP standard date: " << year << "-" << month << std::endl;

    std::cout << "Number of processors: " << omp_get_num_procs() << std::endl;
    std::cout << "Max threads: " << omp_get_max_threads() << std::endl;

    std::cout << "Dynamic threads enabled: "
        << omp_get_dynamic() << std::endl;
}

```

```

std::cout << "Timer resolution: "
          << omp_get_wtick() << " seconds" << std::endl;

std::cout << "Nested parallelism enabled: "
          << omp_get_nested() << std::endl;

std::cout << "Max active levels: "
          << omp_get_max_active_levels() << std::endl;

omp_sched_t sched;
int chunk;
omp_get_schedule(&sched, &chunk);

std::cout << "Schedule type: ";
switch (sched) {
    case omp_sched_static:    std::cout << "static"; break;
    case omp_sched_dynamic:   std::cout << "dynamic"; break;
    case omp_sched_guided:    std::cout << "guided"; break;
    case omp_sched_auto:      std::cout << "auto"; break;
    default:                  std::cout << "unknown"; break;
}
std::cout << "\nChunk size: " << chunk << std::endl;

return 0;
}

```

Файл 3 – Код для вывода параметров OpenMP

7. Результаты вычислительных экспериментов для алгоритма сортировки Шелла

Данные для эксперимента идентичны лабораторной №3: каждая точка на графике была получена путем повторения на 100 сгенерированных массивах размерами 1 млн. элементов. Массивы были одинаковы для всех точек графика.

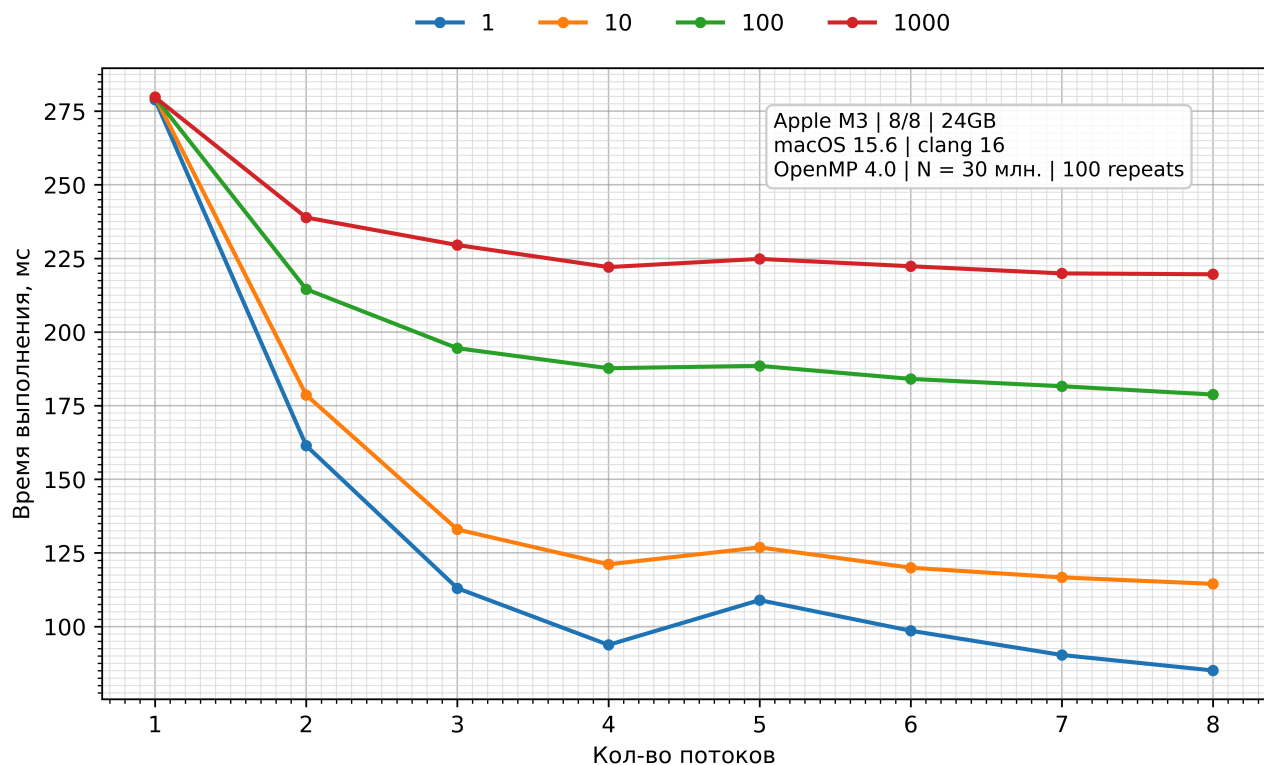


Рисунок 1 – График зависимостей времени обработки от количества потоков в конфигурациях статического (static) режима запуска с 1, 10, 100, 1000 размерами чанков

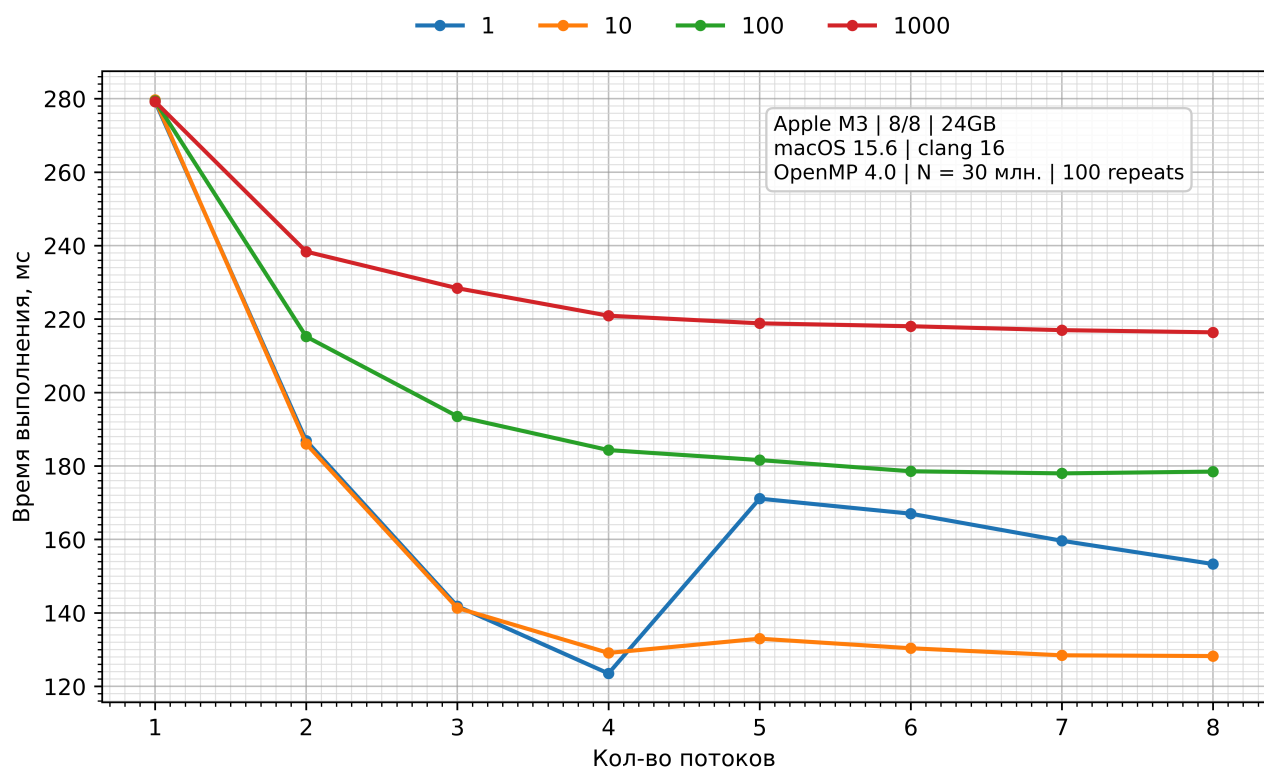


Рисунок 2 – График зависимостей времени обработки от количества потоков в конфигурациях динамического (dynamic) режима запуска с 1, 10, 100, 1000 размерами чанков

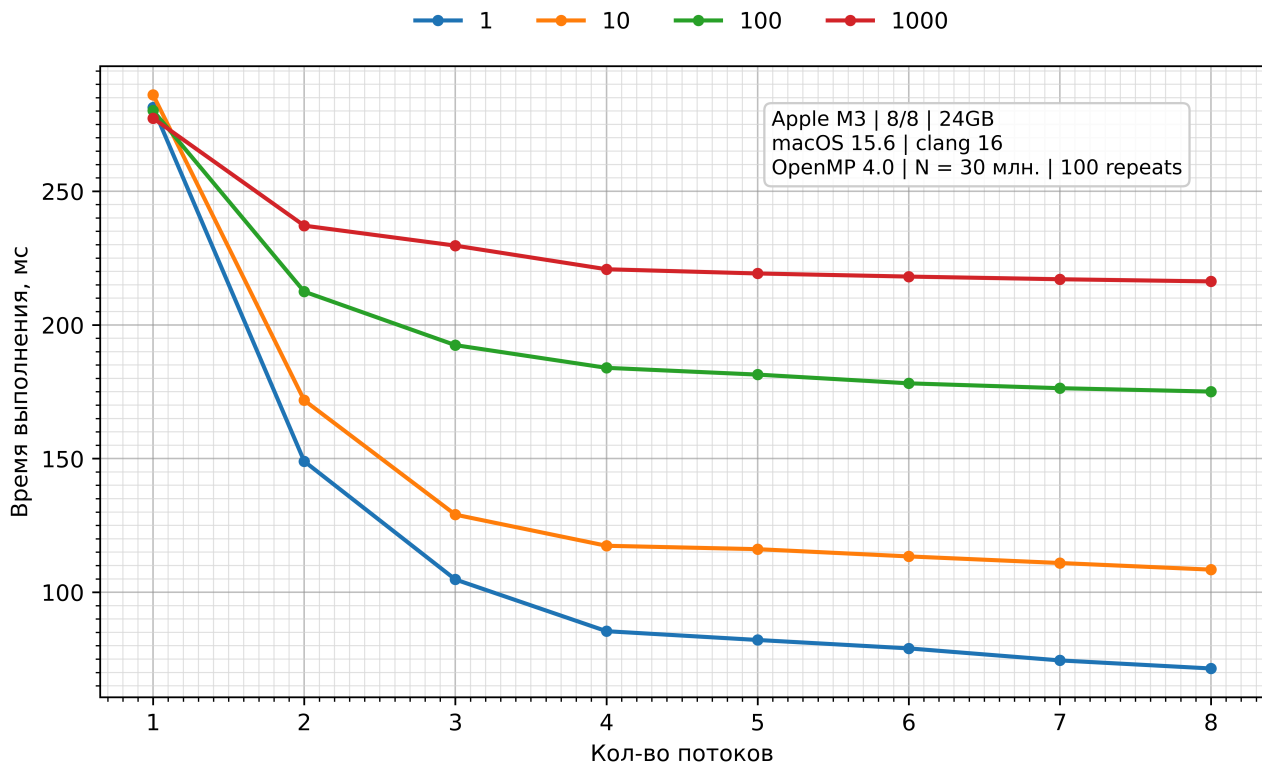


Рисунок 3 – График зависимостей времени обработки от количества потоков в конфигурациях guided режима запуска с 1, 10, 100, 1000 размерами чанков

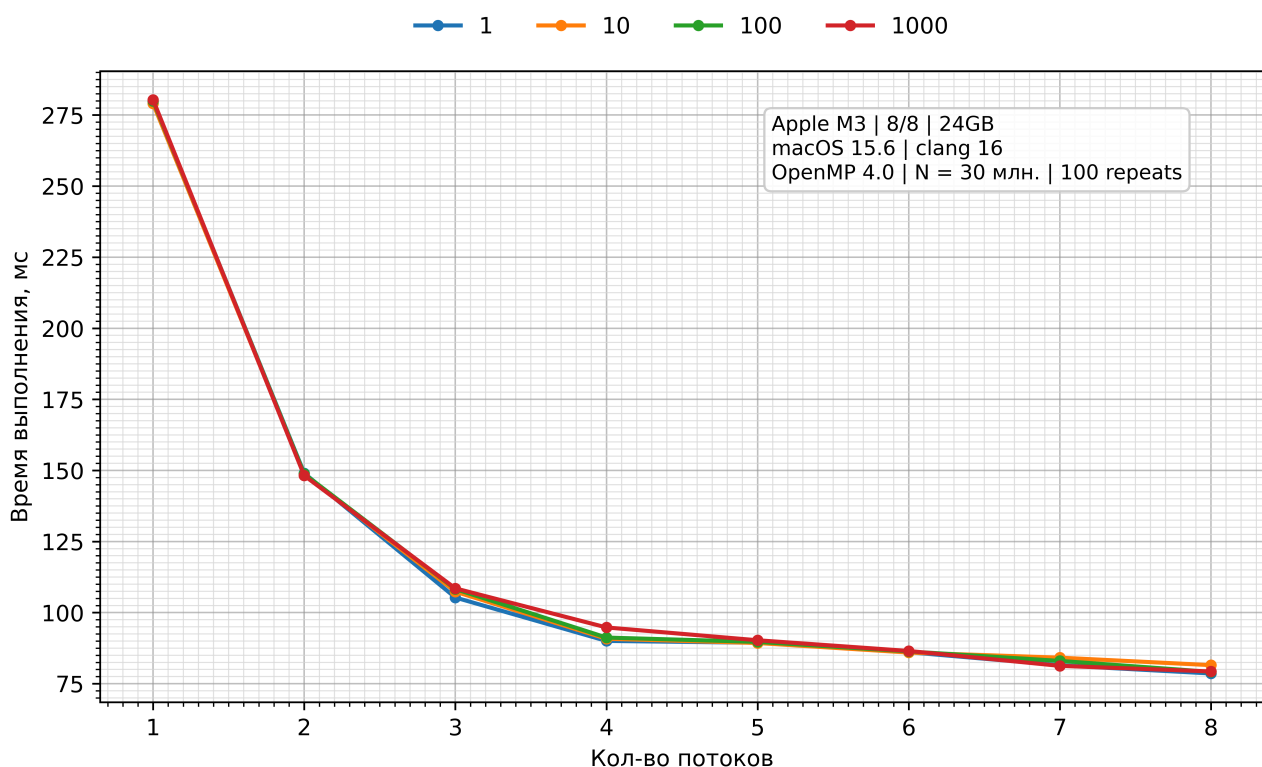


Рисунок 4 – График зависимостей времени обработки от количества потоков в конфигурациях автоматического (auto) режима запуска с 1, 10, 100, 1000 размерами чанков

7.1. Дополнительное задание

Был построен график ускорения для режима guided.

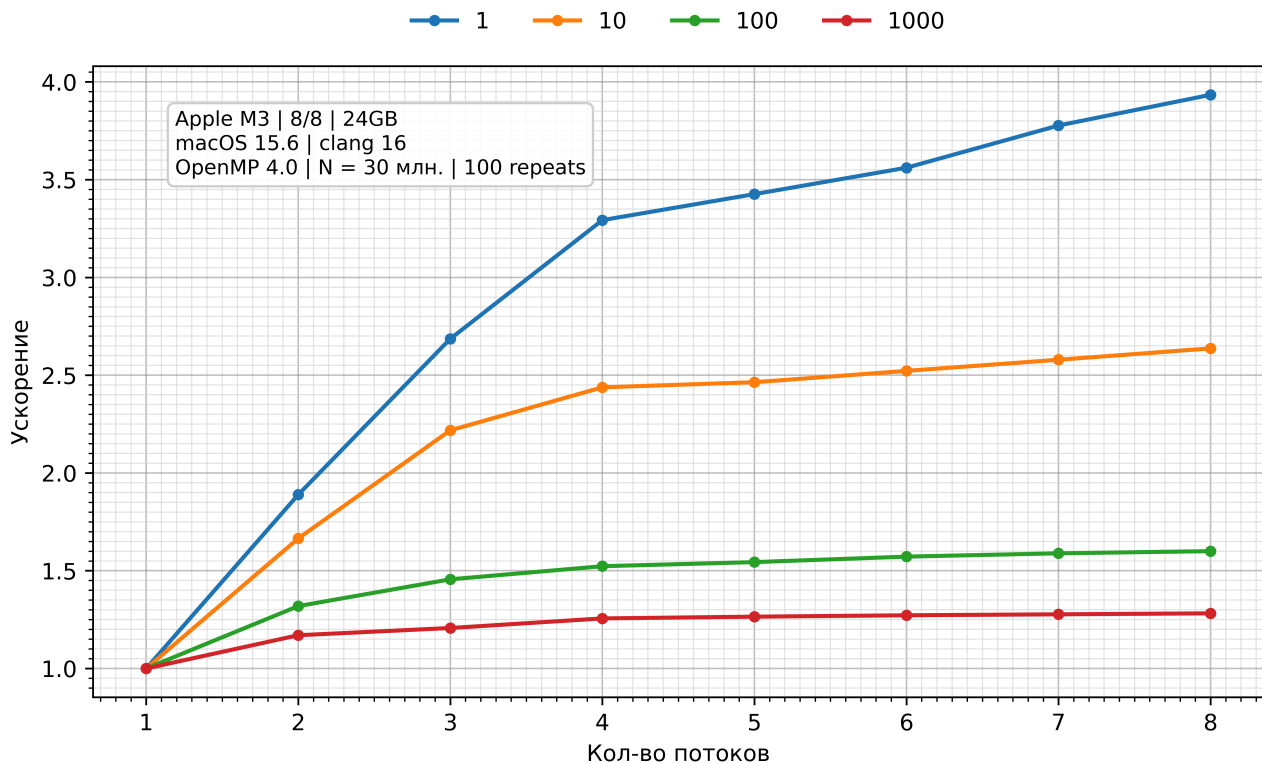


Рисунок 5 – График зависимостей ускорения от количества потоков в конфигурациях guided режима запуска с 1, 10, 100, 1000 размерами чанков для функции сортировки Шелла

Был построен аналогичный график для лабораторной работы №1 для guided режима (ф-я поиска максимума в массиве).

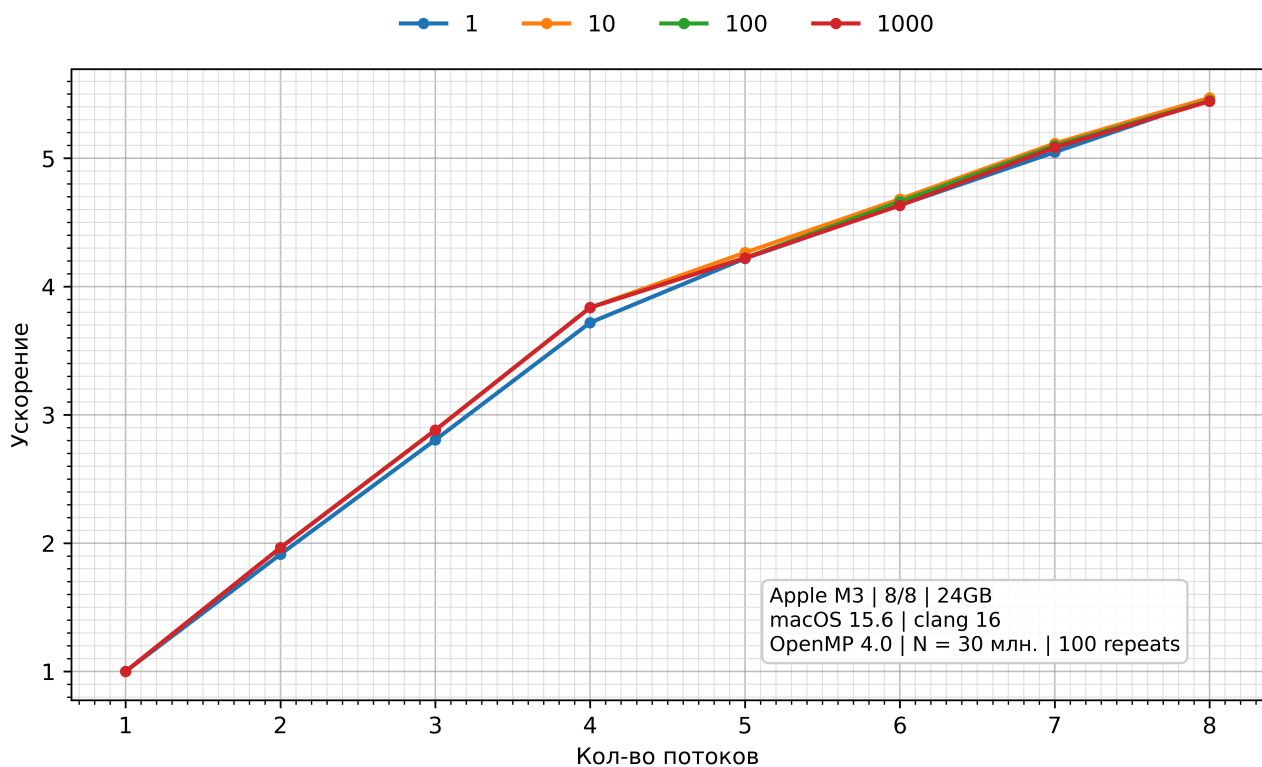


Рисунок 6 – График зависимостей ускорения от количества потоков в конфигурациях guided режима запуска с 1, 10, 100, 1000 размерами чанков для функции поиска максимума в массиве

8. Заключение

Были изучены основные настройки среды OpenMP и связанные с ними возможности. С помощью функций OpenMP были определены доступное число процессоров и потоков, значение опции `dynamic`, разрешение таймера. Также были определены параметры работы с вложенными параллельными областями и уточнены особенности распределения нагрузки в среде OpenMP. Был разработан алгоритм, использующих механизм явных блокировок `omp_set_lock()`.

Для алгоритма, использованного в лабораторной работе №3 был проведен вычислительный эксперимент для разных типов разделения нагрузки и размеров фрагмента (чанка). Результаты проведенного эксперимента показали, что различные политики распределения нагрузки (`static`, `dynamic`, `guided`, `auto`) в сочетании с различными размерами чанков по-разному влияют на производительность параллельного алгоритма сортировки Шелла. В общем случае, увеличение размера чанка отрицательно сказывается на производительности, приводя к росту времени выполнения.

Использование `chunk = 1` обеспечивает максимальную эффективность использования вычислительных ресурсов на всех стадиях алгоритма. Наилучшие результаты были получены при использовании режима `auto`, что можно объяснить тем, что среда выполнения OpenMP самостоятельно подбирает оптимальную стратегию распределения итераций, динамически адаптируясь к текущей структуре нагрузки и особенностям вычислений на каждом этапе работы алгоритма.

Как и в предыдущих лабораторных, на графиках видно резкое увеличение времени выполнения после использования более 4 потоков, что связано с ограниченным числом высокопроизводительных ядер в процессорах Apple M-серии (подробное объяснение см. в заключении лабораторной №1).

Дополнительное задание. Был построен график ускорения для режима `guided`. Был построен аналогичный график для лабораторной работы №1 (ф-я поиска максимума в массиве). На графике видно: для различных размеров `chunk` ускорение идетничное, что отличается от результатов, полученных для функции сортировки Шелла, где ускорение уменьшается с ростом `chunks`.

Различие связано с характером вычислительной нагрузки. При поиске максимума каждая итерация цикла имеет одинаковую стоимость и отсутствуют зависимости по данным, поэтому даже при больших значениях `chunk` в режиме `guided` нагрузка между потоками остаётся равномерной и ускорение практически не меняется. В сортировке Шелла вычислительная сложность итераций неоднородна и присутствуют зависимости по данным, из-за чего увеличение `chunk` приводит к росту дисбаланса между потоками и ухудшению использования памяти. В результате для сортировки Шелла ускорение уменьшается с ростом размера блока.